

UNIVERSIDAD NACIONAL DE COSTA RICA (UNA)

Escuela de Informática • EIF400-II-2026: Paradigmas de Programación

Paradigmas de Programación

Conceptos básicos de la Sesión 02 + Resumen del artículo de Robert W. Floyd

"The Paradigms of Programming" — Conferencia del Premio Turing, 1978 (publicada 1979)

Sesión 02 — 24 de julio

Documento de autoestudio con lenguaje sencillo y muchos ejemplos

Cómo leer este documento

Este material está pensado para alguien que **empieza desde cero**. Cada idea difícil viene acompañada de una explicación en palabras simples y, casi siempre, de un ejemplo de la vida diaria. Está dividido en dos grandes partes:

- **Parte 1 — Conceptos básicos de la sesión de hoy.** Las ideas clave que se van a ver en clase: qué es un paradigma, la metáfora de los "anteojos", el *qué* contra el *cómo*, la relación entre paradigma, lenguaje y compilación, algo de historia y el papel de la IA generativa.
- **Parte 2 — Resumen completo del artículo de Floyd.** Un recorrido detallado, sección por sección, del texto en PDF, explicado en cristiano y con ejemplos.

■ Consejo

No hace falta memorizar. La meta de hoy es tener una **noción intuitiva**: entender de qué se trata todo esto, aunque los detalles se afinen en sesiones futuras (el curso es iterativo).

Conceptos básicos de la Sesión 02

1. ¿Qué es un "paradigma"?

Un **paradigma** es una **manera de ver y resolver problemas**: un conjunto de ideas, costumbres y herramientas mentales que compartimos para atacar cierto tipo de situaciones. No es una regla del lenguaje ni un comando; es una **forma de pensar**.

■ Ejemplo de la vida diaria

Imagina que tienes que llegar a otra ciudad. Puedes pensar el viaje de varias formas:

- **Como conductor:** piensas en calles, giros, gasolina y semáforos.
- **Como pasajero de avión:** piensas en horarios, boletos y puertas de embarque.

El destino es el mismo, pero cada "forma de pensar" te hace fijarte en cosas distintas y usar herramientas distintas. Eso es un paradigma: **el enfoque con el que abordan el problema**.

En programación, un paradigma es el enfoque con el que diseñamos programas. Ejemplos que oirás en el curso: **imperativo, declarativo, orientado a objetos, funcional (FP)**, entre otros. Cada uno propone una manera distinta de organizar las ideas.

2. La metáfora de los "anteojos"

Un paradigma funciona como un par de **anteojos** (lentes) que te pones para mirar un problema. Según el lente que uses, **ves unas cosas con claridad y otras se te esconden**.

■ La idea en una frase

Cambiar de paradigma es como cambiar de lentes: **el problema no cambia, pero lo que notas de él sí**. Un buen programador tiene varios pares de anteojos y sabe cuál ponerse en cada momento.

■ Cuidado

Ningún lente sirve para todo. Un mismo par de anteojos que te ayuda con un problema puede estorbarte en otro. Por eso conviene conocer varios paradigmas, no casarse con uno solo.

3. El QUÉ (declarativo) frente al CÓMO (imperativo)

Esta es una de las distinciones más importantes de la sesión. Hay dos grandes maneras de decirle a la computadora lo que queremos:

Estilo IMPERATIVO (el CÓMO)

Estilo DECLARATIVO (el QUÉ)

Describes paso a paso las instrucciones que debe seguir la máquina para llegar al resultado.	Describes qué resultado quieres , y dejas que el sistema se encargue de cómo lograrlo.
"Haz esto, luego esto, luego aquello."	"Quiero esto; arréglatelas para conseguirlo."
Ejemplos: C, la mayoría del código Java clásico, un bucle for.	Ejemplos: SQL, HTML, expresiones de estilo funcional.

■ Ejemplo cotidiano: preparar un café

Manera imperativa (el CÓMO): "Pon agua a hervir, muele el café, coloca el filtro, vierte el agua despacio, espera 4 minutos, sirve." Das cada paso.

Manera declarativa (el QUÉ): "Quiero un café mediano con leche." Se lo dices al barista y él decide cómo hacerlo. Tú solo describes el resultado.

■ Ejemplo con código: sumar los números de una lista

Imperativo — explicas cada paso del recorrido:

```
total = 0
for (i = 0; i < lista.length; i++) {
  total = total + lista[i];
}
```

Declarativo — solo dices "súmalos":

```
total = lista.reduce((a, b) => a + b, 0);
```

Los dos dan el mismo resultado. La diferencia es **cuánto detalle del "cómo" tienes que escribir tú**.

4. El paradigma como abstracción para modelar (Floyd–Kuhn, 1974)

La palabra "paradigma" no nació en la informática. El historiador de la ciencia **Thomas Kuhn** la popularizó para explicar cómo avanza la ciencia: durante un tiempo todos comparten un mismo "modelo mental" (paradigma); cuando ese modelo ya no alcanza, ocurre una **revolución** y se cambia por uno nuevo (por ejemplo, pasar de creer que el Sol gira alrededor de la Tierra a lo contrario).

Robert Floyd tomó esa idea y la trajo a la programación: los programadores también trabajamos con paradigmas, y progresamos cuando **inventamos, mejoramos y compartimos** paradigmas nuevos. Aquí un paradigma es una **abstracción**: un modelo simplificado que nos deja pensar el problema sin ahogarnos en los detalles.

■ ¿Qué es "abstraer"?

Abstraer es **quedarse con lo importante y esconder lo demás**. Un mapa del metro es una abstracción: no muestra las calles reales ni las distancias exactas, solo lo que necesitas para saber en qué estación bajarte. Es más útil *precisamente porque* esconde detalles.

5. Paradigma, lenguaje de programación y compilación

Conviene no confundir tres cosas distintas:

- **Paradigma**: la forma de pensar el problema (la idea). No se "instala", se aprende.
- **Lenguaje de programación**: el idioma concreto con el que escribimos el programa (Java, JavaScript, Python...). Un lenguaje suele **favorecer** ciertos paradigmas y **dificultar** otros.
- **Compilación**: el proceso que **traduce** tu código, escrito en un lenguaje que las personas entienden, a instrucciones que la máquina puede ejecutar.

■ Una analogía para los tres

Paradigma = tu estilo de contar una historia (drama, comedia...).

Lenguaje = el idioma en que la escribes (español, inglés...).

Compilación = el traductor que la convierte a otro idioma para que otro público la entienda.

La misma historia (mismo paradigma) puede escribirse en varios idiomas (lenguajes), y luego traducirse (compilarse) para que la "lea" la máquina.

■ Idea clave de Floyd (adelanto de la Parte 2)

Un lenguaje **apoya** un paradigma cuando hace **cómodo** programarlo, y lo **apoya débilmente** cuando lo hace posible pero incómodo. Un buen lenguaje debería apoyar los paradigmas que su comunidad realmente usa.

6. Algunas referencias históricas

- **Programación estructurada** (Dijkstra, Wirth, Parnas, años 60–70): ordenar el programa descomponiéndolo en partes cada vez más simples, evitando el caos del "salta para aquí, salta para allá".
- **Thomas Kuhn (1962)**: introdujo la idea de "paradigma" y de "revoluciones" en la ciencia.
- **Robert Floyd (Premio Turing 1978)**: llevó esa idea a la programación; su conferencia es justo el texto de la Parte 2.
- **Lisp y la familia Algol**: dos tradiciones distintas de lenguajes que Floyd compara en su charla.

El **Premio Turing** es el reconocimiento más importante en computación, algo así como el "Premio Nobel" del área.

7. La IA generativa y la programación: ¿hoy y futuro?

Hoy existen herramientas de **inteligencia artificial generativa** (como los asistentes que escriben código a partir de instrucciones en lenguaje natural). Esto conecta con una vieja promesa que Floyd menciona en 1979: la "**programación automática**", la idea de que algún día las máquinas escribirían los programas por nosotros.

La reflexión para hoy es doble:

- **Oportunidad:** la IA puede ayudarnos a escribir código más rápido y a aprender paradigmas nuevos viendo ejemplos.
- **Advertencia (muy en la línea de Floyd):** la herramienta no reemplaza el **entender**. Si no dominas los paradigmas, no sabrás si lo que la IA produjo está bien pensado. La IA cambia el *cómo*, pero el *qué* y el buen criterio siguen siendo tuyos.

8. Práctica inicial con Java y JavaScript

La sesión también arranca la práctica con **Java** y **JavaScript (JS)** usando entornos minimalistas (lo mínimo necesario para programar, sin complicaciones). La meta es ver, con las manos en el código, cómo un mismo problema se puede resolver con distintos **patrones** y con **programación funcional (FP)**.

■ Java y JS lado a lado (mismo paradigma funcional)

Duplicar cada número de una lista, al estilo declarativo/funcional:

```
// JavaScript
const dobles = nums.map(n => n * 2);

// Java
List<Integer> dobles = nums.stream()
    .map(n -> n * 2)
    .toList();
```

Aunque los idiomas (lenguajes) son distintos, la **forma de pensar** es la misma: "transforma cada elemento", sin escribir el bucle a mano. Ese es el paradigma funcional en acción.

Resumen completo del artículo de Robert W. Floyd

"The Paradigms of Programming" — Conferencia del Premio Turing, 1978 (publicada en Communications of the ACM, 1979).

En pocas palabras: de qué trata el texto

Floyd sostiene que lo más valioso que un programador puede aprender **no son** los comandos de un lenguaje ni una lista de algoritmos memorizados, sino los **paradigmas**: las formas de pensar y atacar problemas. Y pide tres cosas: que los programadores **afinen** sus métodos, que los profesores los **enseñen de forma explícita**, y que los lenguajes los **apoyen** de verdad.

1. El punto de partida: la programación estructurada

Floyd abre con el ejemplo de paradigma más conocido de su época: la **programación estructurada**. Explica que tiene dos fases:

- **Diseño de arriba hacia abajo ("top-down") o refinamiento paso a paso**: agarras el problema grande y lo partes en unos pocos subproblemas más simples; luego partes esos otra vez, y así hasta que cada pedazo es tan sencillo que ya lo puedes resolver directo.
- **Construcción de abajo hacia arriba con "niveles de abstracción"**: empiezas por las piezas concretas de la máquina y vas construyendo piezas cada vez más abstractas encima. A esto también se le llama **ocultamiento de información** ("information hiding").

■ El ejemplo que usa Floyd: resolver ecuaciones

Para resolver un sistema de ecuaciones lineales, primero lo divide en dos etapas grandes: "triangularizar" las ecuaciones y luego hacer "sustitución hacia atrás". Cada etapa se vuelve a partir en pasos más chiquitos hasta llegar al algoritmo completo. No necesitas saber la matemática: lo importante es la **estrategia de partir el problema grande en partes manejables**.

Floyd aclara que la programación estructurada **no resuelve todo**. Siguen siendo necesarios otros paradigmas más especializados, como **"divide y vencerás"** (partir el problema en mitades, resolverlas y juntarlas) o **"ramificación y poda"** (branch-and-bound). Pero este paradigma sí **amplía tu capacidad**: te permite construir programas demasiado complicados para hacerlos sin un método.

2. El diagnóstico: la "depresión del software"

Floyd dice que el estado de la programación es pobre por varias razones: nos faltan paradigmas, no conocemos bien los que ya existen, los enseñamos mal, y nuestros lenguajes no los apoyan. Cita a Robert Balzer, quien describía el software como poco confiable, entregado tarde, caro e

ineficiente.

■ El chiste que hace Floyd

A la famosa "crisis del software" de la que se hablaba desde hacía 10 o 15 años, Floyd propone llamarla mejor "**depresión del software**": si llevamos tanto tiempo igual de mal, ya no es una crisis pasajera, es un estado crónico.

3. La conexión con Thomas Kuhn y la ciencia

Aquí Floyd trae al historiador **Thomas Kuhn**. Kuhn decía que la ciencia avanza a saltos: la comunidad comparte un paradigma, y cuando este ya no alcanza, llega una **revolución** que lo reemplaza. Floyd observa que en la computación pasa algo parecido y saca varias ideas:

- Los libros de texto suelen dar a entender que "programar" es **solo** conocer algoritmos y reglas del lenguaje. Floyd dice que eso deja fuera lo más importante: los paradigmas.
- Existen **comunidades** de programadores, cada una con su propio "idioma" y sus paradigmas favoritos: hay escuelas de Lisp, de APL, de Algol, etc. El lenguaje que usas **moldea** tu forma de pensar.
- Cuando llega un paradigma nuevo, las escuelas viejas van desapareciendo. Y bromea: en la computación, a los que se aferran a lo viejo nadie los expulsa... "probablemente terminan siendo gerentes de desarrollo de software".

4. Los paradigmas hay que inventarlos y compartirlos

Floyd cuenta historias reales para mostrar que **el progreso viene de inventar paradigmas nuevos**:

- **John Cocke** resolvió un problema difícil de compiladores (analizar el lenguaje, "parsing") con un algoritmo rápido y simple, basado en la **programación dinámica**: resolver primero todos los casos pequeños y usarlos para armar los grandes. Con ese enfoque, el problema se volvió casi trivial.
- **El propio Floyd** resolvió otro problema imaginando una "organización jerárquica de trabajadores" que se contratan y despiden entre sí, y luego simulándola. Eso lo llevó a usar **corrutinas recursivas** como estructura de control. Descubrió que otros habían inventado, por su cuenta, la misma idea.

■ MYCIN: mejorar un paradigma (sistemas basados en reglas)

MYCIN era un programa que diagnosticaba infecciones y recomendaba medicinas. Estaba hecho con muchas **reglas independientes**: "si se cumple tal condición, entonces haz tal cosa".

Un programa llamado TEIRESIAS le agregó una capacidad: cuando MYCIN daba un resultado equivocado, **rastreaba hacia atrás** qué regla tuvo la culpa. Así, un médico —sin saber programar— podía corregir el sistema. Todo eso se pudo hacer **gracias a elegir el paradigma de reglas**, no un montón de "if" enredados.

5. Cómo crece un programador: ampliar tu repertorio

Si el **arte general** de programar avanza inventando paradigmas, el **programador individual** avanza **coleccionando** más paradigmas. Floyd comparte su técnica personal, muy práctica:

1. Resuelve un problema difícil.
2. Vuelve a resolverlo **desde cero**, recordando solo la idea clave.
3. Repite hasta que la solución sea lo más clara y directa posible.
4. Busca la **regla general** que te habría llevado a esa solución **a la primera**. Esa regla suele tener valor permanente: es un paradigma nuevo para tu colección.

Floyd también dice que **leer programas de otros** ayuda, pero con un límite: normalmente tus colegas piensan parecido a ti. Por eso recomienda **exponerte a estilos ajenos**. Él, que venía de amar lenguajes "ricos" como Algol, al visitar el MIT quedó impresionado por el poder de **Lisp**, donde el código y los datos tienen la misma forma y un programa puede manipular a otros programas.

■ Frase para recordar

Floyd cita una pared del MIT: "Preferiría escribir programas que me ayuden a escribir programas, que escribir programas." La idea de crear herramientas que potencian tu propio trabajo. También comenta que las reglas de un lenguaje como Fortran se aprenden en horas, pero sus **paradigmas** tardan mucho más en aprenderse... y en **desaprenderse**.

6. Cuando el lenguaje ayuda... y cuando estorba

Floyd introduce una distinción muy citada:

- Un lenguaje **apoya** un paradigma cuando lo hace **cómodo** de usar.
- Un lenguaje lo **apoya débilmente** cuando lo hace posible, pero **incómodo** (te toca dar muchas vueltas).

Y da dos ejemplos concretos de cuando el lenguaje estorba:

■ Ejemplo 1: la asignación simultánea (lobos y conejos)

Imagina una simulación de depredador-presa: lobos (W) y conejos (R). Cada periodo, los nuevos valores dependen de los valores **viejos** de ambos. Un principiante escribe:

```
W := f(W, R);  
R := g(W, R);
```

¡Error! Al calcular R ya se usó el W **nuevo**, no el viejo. Para arreglarlo hay que inventar una variable temporal:

```
TEMP := f(W, R);  
R     := g(W, R);  
W     := TEMP;
```

Floyd dice: el principiante **tiene razón** en molestarse. Actualizar varias cosas "a la vez" es un paradigma muy común (lo llama **asignación simultánea**), y casi ningún lenguaje lo ofrece de forma directa. Nos obliga al truco mecánico y propenso a errores de la variable temporal.

■ Ejemplo 2: el texto a 30 caracteres por línea (corrutinas)

Otra tarea de apariencia inocente: leer texto, quitar espacios de más y reimprimirlo a 30 caracteres por línea sin cortar palabras. Suena fácil, pero es **sorprendentemente difícil** en la mayoría de lenguajes, porque los bucles de entrada y de salida no encajan uno dentro del otro.

La forma natural de resolverlo es con **corrutinas**: tres procesos que se comunican (uno lee, otro transforma, otro imprime). Pero casi ningún lenguaje de la época ofrecía corrutinas cómodas. Resultado: los novatos tardaban tres o cuatro veces más de lo esperado y terminaban con código enredado.

Floyd remata: incluso la **programación estructurada** está mal apoyada por muchos lenguajes. A veces uno ni siquiera puede escribir las partes del programa en el orden en que las diseñó, o se ve obligado a hacer variables "globales" que rompen el ocultamiento de información.

7. Un nivel aún más alto: jerarquías de lenguajes

Existe un paradigma todavía más ambicioso que la programación estructurada: construir una **jerarquía de lenguajes**. Escribes en un lenguaje de muy alto nivel (que maneja ideas abstractas) y ese se traduce a uno de más bajo nivel, y así sucesivamente. Es como tener un idioma especializado montado encima de otro más básico.

El problema, señala Floyd, es que los lenguajes de bajo nivel no apoyan bien estas "superestructuras": por ejemplo, cuando hay un error, el mensaje habla del programa **traducido** (el de abajo) y no del que tú realmente escribiste (el de arriba), y entonces no se entiende.

Su conclusión para el diseño de lenguajes: antes de diseñar un lenguaje, primero hay que **enumerar los paradigmas** que su comunidad usará. Y añade que el **entorno completo** (editores,

sistema de archivos, mensajes de error) también debería apoyar los paradigmas —celebra, por ejemplo, los editores que "entienden" la estructura del programa que editas.

8. Qué y cómo enseñar programación

Floyd critica la "obsesión por la forma sobre el contenido". Dice que si le preguntas a un profesor qué enseña y responde "Pascal" o "Fortran", en realidad está enseñando **gramática, reglas y algoritmos terminados**, y deja que el alumno descubra solo **cómo diseñar**. Su propuesta: **enseñar explícitamente métodos de diseño** en todos los niveles. Da ejemplos concretos de paradigmas enseñables:

- **PROMPT_READ_CHECK_ECHO**: un patrón estándar para pedir datos al usuario: pregunta, lee, verifica que el dato sea válido (si no, vuelve a pedirlo) y confirma. Enseña de paso una idea importante: **cada parte del programa debe protegerse de datos para los que no fue diseñada**.
- **Generar / filtrar / acumular**: un patrón enseñado en el MIT. Muchos problemas que parecen distintos son en el fondo lo mismo: **enumerar** los elementos de un conjunto, **filtrar** un subconjunto y **acumular** un resultado. El alumno solo pone las tres piezas.
- **El paradigma de la máquina de estados**: representar la situación con un conjunto de variables y una función que dice cómo pasar de un estado al siguiente (la simulación de lobos y conejos es un caso).

■ Generar / filtrar / acumular en un ejemplo simple

"Suma los precios de los productos que cuestan más de 100."

- **Generar**: recorrer todos los productos.
- **Filtrar**: quedarte solo con los que cuestan más de 100.
- **Acumular**: ir sumando esos precios.

Una vez que reconoces este patrón, muchísimos problemas distintos se resuelven "rellenando" esas tres partes.

Floyd añade que muchos algoritmos clásicos son en realidad **casos** de un paradigma más amplio: el ordenamiento por mezcla ("merge sort") es un caso de **divide y vencerás**; la eliminación gaussiana es recursión convertida en iteración, etc. Ante cada algoritmo clásico conviene preguntarse: "**¿Cómo podría haber inventado esto yo mismo?**" y recuperar así el paradigma que hay detrás.

9. El mensaje final de Floyd

Floyd cierra con tres recomendaciones, una para cada tipo de lector:

Para el programador	Para el profesor	Para el diseñador de lenguajes
---------------------	------------------	--------------------------------

Dedica parte de tu día a examinar y afinar tus propios métodos. Es una inversión a largo plazo.	Identifica los paradigmas que usas y enséñalos de forma explícita; le servirán a tus alumnos aunque el lenguaje de moda cambie.	Si tu lenguaje no apoya los paradigmas que la gente usa (o no la deja extenderlo), no aporta nada nuevo. Vuélvete un estudiante serio del proceso de diseño.
---	---	--

Mini-glosario (por si acaso)

Paradigma	una forma de pensar y resolver problemas; el "lente" con que miras.
Declarativo (el QUÉ)	dices qué resultado quieres, no los pasos.
Imperativo (el CÓMO)	dices los pasos, uno por uno.
Abstracción	quedarse con lo esencial y esconder los detalles.
Programación estructurada	partir el problema en partes cada vez más simples.
Divide y vencerás	partir en mitades, resolver cada una y juntar.
Corrutinas	procesos que se van pasando el control entre sí para colaborar.
Asignación simultánea	actualizar varias variables "a la vez" con sus valores viejos.
Sistema basado en reglas	programa hecho de muchas reglas "si... entonces...".
Compilación	traducir el código a instrucciones que la máquina ejecuta.
Apoyar un paradigma	que el lenguaje lo haga cómodo (no solo posible).

Documento de autoestudio — EIF400 Paradigmas de Programación, Sesión 02 (24/07). Resumen del artículo de Robert W. Floyd, "The Paradigms of Programming" (Turing Award, 1978).